TRIVIA FIESTA

Student Number: 10911878

Module Code: COMP1004

Deadline: 30 Apr 2026

Module Leader: Dr Vasilios Kelefouras

Dr Lauren Ansell

Contents

1. Introduction 2
2. Background and Existing Systems 2
3. Methodology 3
4. Analysis 3
4.1 Functional Requirements 4
4.2 Non- Functional Requirements 4
4.3 User Stories 5
4.4 Legal, Social, Ethical and Professional Considerations 6
4.5 Sprint Reflection 7
5. System Design 8
5.1 Overview 8
5.2 Architecture Design 8
5.3 Data Design (JSON Structure) 9
5.4 User Interface Design 9
5.5 Interaction and Navigation Design 9
5.6 Design Decisions and Justification 10
6. Implementation 10
6.1 Overview 10
6.2 Application State Management 10
6.3 DOM Helper Utility 11
6.4 Question Handling and Logic 12
6.5 Custom Quiz Upload and Validation 14
6.6 Local Storage Leaderboard 15
6.7 JSON Score Export 15
7. Testing 16
8. Evaluation and Conclusion 16
9. References 18
APPENDIX A – Sprint Plan and Task Tracker 19
APPENDIX B – Data Design 22
APPENDIX C – Test Plan 24
APPENDIX D – UML Diagrams 30
APPENDIX E - User Interface and Validation Feedback 34
APPENDIX F – AI Declaration Form 36

1. Introduction

Trivia Fiesta is a web-based quiz application developed as part of the COMP1004 module, designed to allow users to test their knowledge across multiple categories and difficulty levels. The application is implemented as a Single Page Application (SPA) using HTML, CSS, and JavaScript, with all functionality executed client-side in accordance with module requirements prohibiting backend technologies.

Development followed the Software Development Life Cycle (SDLC), providing a structured approach to planning, design, implementation, and evaluation. An Agile methodology using Scrum principles was adopted, dividing the project into iterative sprints to enable continuous feedback and refinement (Pressman, 2014; Schwaber and Sutherland, 2020).

The system provides a customisable quiz experience in which users can select their preferred language (English or Greek), category, and difficulty level. Questions are dynamically loaded from structured JSON data, fulfilling the module requirement for JSON input. Additional features include a countdown timer, immediate answer feedback, a locally stored leaderboard, custom quiz upload via JSON, and score export — demonstrating both data input and output capabilities.

Key design decisions include varying question count by difficulty level and implementing a centralised state object to manage application data consistently across views, contributing to a responsive and reliable user experience within a fully client-side architecture.

2. Background and Existing Systems

Quiz-based applications are widely used in both educational and entertainment contexts. Platforms such as Kahoot (Kahoot!, n.d.), Wayground (Wayground, 2025.), and Sporcle (Sporcle, n.d.) offer structured quiz environments with multiple choice questions, scoring systems, and leaderboards, to enhance engagement.

Many existing platforms rely on server-based architectures requiring internet connectivity, user accounts, or instructor-led sessions. Kahoot is primarily designed for classroom use, while Wayground focuses on structured assessment, limiting flexibility for independent use.

In contrast, Trivia Fiesta operates entirely as a client-side SPA using HTML, CSS, and JavaScript, with no backend technologies. All processing occurs within the browser, removing the need for network connectivity or server infrastructure. LocalStorage is used to persist leaderboard data, enabling score tracking without a database.

The application also extends beyond basic functionality through bilingual support (English and Greek), custom quiz upload via JSON, and JSON score export, allowing users to create, load, and reuse quiz data externally.

3. Methodology

The development of Trivia Fiesta followed a structured Software Development Life Cycle (SDLC), providing a systematic approach to planning, design, implementation, testing, and evaluation. An Agile methodology using Scrum principles was adopted to support iterative development, dividing the project into a series of manageable sprints, each focused on implementing specific functionality derived from the product backlog (Pressman, 2014; Schwaber and Sutherland, 2020).

The project was organised into nine sprints, progressing from project initiation and requirements analysis through core implementation, feature enhancement, testing, and finalisation. At the start of each sprint, tasks were prioritised based on their importance to core functionality, forming the sprint backlog and guiding development during each iteration. A task tracker was maintained to document sprint goals, individual tasks, progress, and completion status, reflecting key Scrum practices of backlog management and task prioritisation. The full sprint plan and task tracker are provided in Appendix A.

Regular stand-up meetings were conducted approximately every two weeks with a supervising lecturer, providing opportunities to review progress, discuss challenges, and receive feedback. This aligns with Agile principles of continuous improvement, enabling issues to be identified and addressed early rather than accumulating across sprints.

4. Analysis

While the core functional requirements remained stable throughout development, several features were refined during implementation. The custom quiz upload functionality, for example, evolved from a simple file input into a fully validated feature with structured user feedback. The leaderboard was similarly extended to support JSON export to meet module requirements.

Non-functional requirements such as usability and reliability also became more prominent during later stages of development.

4.1 Functional Requirements

Functional requirements define the core features and behaviour of the system. These were identified during the planning phase and form the basis of the application's functionality. Table 1 presents all functional requirements with their assigned priority.

Table 1: Functional Requirements

ID	Requirement	Description	Priority
FR1	Language Selection	The system must allow users to select between English and Greek	High
FR2	Category Selection	The system must allow users to choose a quiz category	High
FR3	Difficulty Selection	The system must allow users to select a difficulty level	Medium
FR4	Question Display	The system must display multiple choice questions	High
FR5	Answer Selection	The system must allow users to select an answer	High
FR6	Answer Feedback	The system must provide immediate feedback on answers	Medium
FR7	Score Calculation	The system must calculate user's final score	High
FR8	Score Display	The system must display the final score at the end of the quiz	High
FR9	Name Input	The system must allow users to enter their name for score recording	Medium
FR10	Leaderboard Storage	The system must store scores using localStorage	Medium
FR11	Leaderboard Display	The system must display top scores in a leaderboard	Medium
FR12	JSON Score Export	The system must allow users to export scores as a JSON file	High
FR13	Custom Quiz Upload	The system must allow users to upload quiz files in JSON format	High
FR14	JSON Validation	The system must validate uploaded quiz files before use	High
FR15	Navigation Controls	The system must provide navigation (next,back,play again)	High
FR16	Countdown Timer	The system must include a timer for each question	Medium
FR17	Audio Feedback	The system must provide sound feedback for answers	Low
FR18	Theme Toggle	The system must allow switching between light and dark mode	Low

4.2 Non- Functional Requirements

Non-functional requirements define the quality attributes and constraints of the system. Table 2 presents all non-functional requirements with their assigned priority.

Table 2: Non-Functional Requirements

ID	Requirement	Description	Priority
NFR1	Technology Constraint	The system must be built using HTML, CSS and JavaScript only	High
NFR2	SPA Architecture	The system must function as a Single Page Application	High
NFR3	Browser Compatibility	The system must run on modern browsers (Chrome, Edge, Firefox)	High
NFR4	Performance	The system should respond quickly to user interactions	Medium
NFR5	Usability	The interface should be clear, structured, and easy to navigate	High
NFR6	Accessibility	The system should support bilingual interaction (English and Greek)	Medium
NFR7	Data Persistence	The system should store data locally using localStorage	Medium
NFR8	Error Handling	The system should handle invalid inputs gracefully	High
NFR9	Maintainability	The code should be modular and well-structured	Medium
NFR10	Consistency	The UI should remain consistent across all views	Medium

4.3 User Stories

User stories were used to define system behaviour from both user and developer perspectives. Each story follows the format:
“As a [user], I want [feature], so that [benefit]”

User-Focused Stories

- As a user, I want to select a language so that I can use the application in English or Greek.
- As a user, I want to choose a quiz category so that I can answer questions related to my interests.
- As a user, I want to answer multiple choice questions so that I can test my knowledge.
- As a user, I want to select a difficulty level so that the quiz matches my ability.
- As a user, I want a countdown timer so that the quiz is more engaging and challenging.
- As a user, I want immediate feedback on my answers to that I understand my performance.
- As a user, I want to see my final score so that I can evaluate my results.

- As a user, I want to save my score so that it appears on the leaderboard.
- As a user, I want to export scores as a JSON file so that I can store or reuse results externally.
- As a user, I want to upload my own quiz file so that I can create custom quizzes.

Developer-Focused Stories

- As a developer, I want the application to function as a Single Page Application so that views update dynamically without page reloads.
- As a developer, I want quiz data to be stored in JSON format so that it is structured and easy to manage.
- As a developer, I want uploaded quiz files to be validated so that invalid data does not break the system.
- As a developer, I want leaderboard data to be stored in localStorage so that scores persist between sessions.
- As a developer, I want reusable helper functions so that UI rendering remains consistent and secure.

4.4 Legal, Social, Ethical and Professional Considerations

The development of Trivia Fiesta involved consideration of several legal, social, ethical, and professional factors. From a legal perspective, the application does not collect or transmit personal data, as all user information is stored locally using browser localStorage, avoiding data protection concerns under GDPR.

Social considerations were addressed through bilingual support (English and Greek), improving accessibility for a wider range of users. The interface was designed to be clear and readable, supporting users with varying levels of technical experience.

Ethically, the application was designed to ensure a fair and transparent user experience. Clear feedback is provided when answering questions or uploading invalid quiz files, and neutral, non-judgemental language was used throughout to support a positive experience.

From a professional perspective, the project followed structured development practices including version control, Agile methodology,

and regular supervisor reviews, reflecting industry standards for software development.

4.5 Sprint Reflection

The nine sprints progressed from project initiation and requirements analysis through to implementation, feature enhancement, testing, and finalisation. Sprints 1 and 2 covered project scoping, requirements definition, and JSON data structure planning. Sprints 3 and 4 established the core application structure, including SPA navigation, question rendering, and basic scoring, ensuring a functional foundation. Sprint 5 addressed mid-project deliverables, while Sprints 6 and 7 introduced advanced features including the countdown timer, sound effects, theme toggle, and custom quiz upload with validation. Sprints 8 and 9 focused on testing, bug fixing, and finalisation. A full breakdown of sprint tasks is provided in Appendix A.

Several features evolved during development. The custom quiz upload feature was extended to include structured validation and user feedback to prevent invalid input. The leaderboard was enhanced with JSON score export to meet data output requirements, and bilingual support introduced additional complexity in maintaining consistent translations.

A notable challenge arose during usability testing, where navigating backwards allowed previously answered questions to be reattempted. This was traced to inconsistent state management and resolved by ensuring answers were persistently stored and validated before interaction. This highlighted the importance of iterative testing, as the issue only surfaced through real user interaction.

Overall, the sprint-based approach enabled continuous refinement, allowing both technical and usability issues to be identified and addressed progressively.

5. System Design

5.1 Overview

The application follows a Single Page Application (SPA) architecture, where content is dynamically rendered without full page reloads, improving responsiveness and user experience. (Flanagan, 2020)

The design follows a modular structure, separating concerns between data handling, application logic, and user interface rendering. This improves maintainability and allows features to be developed and tested independently.

UML diagrams illustrating system architecture and behaviour are provided in Appendix D, with a representative sequence diagram shown in Figure 1.

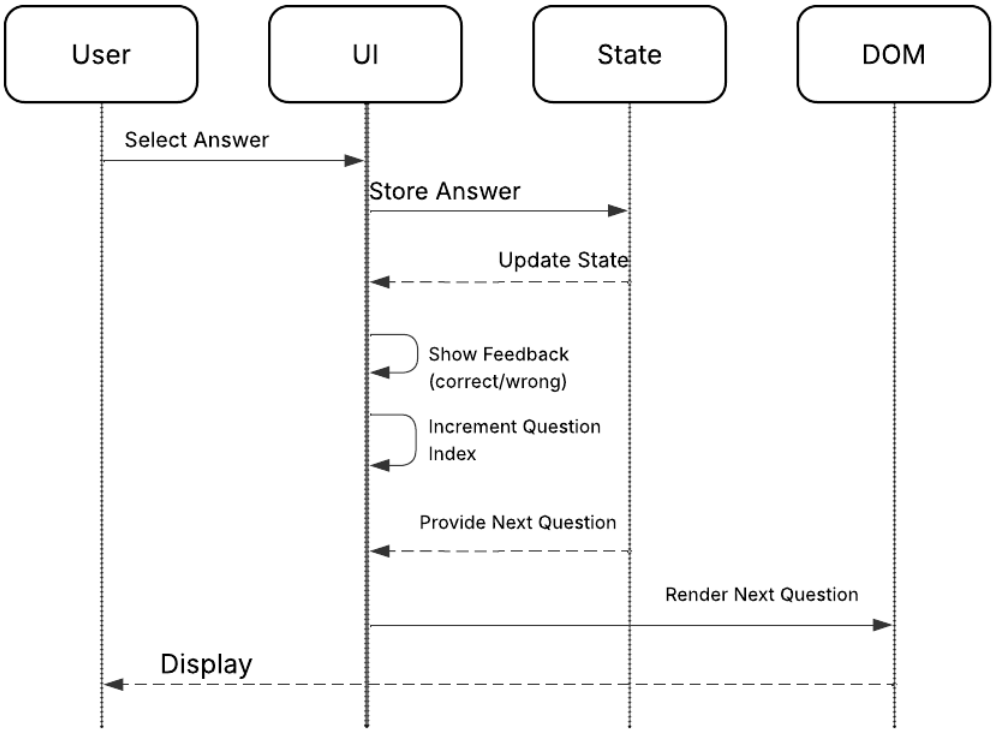

Figure 1 Sequence diagram illustrating quiz interaction flow

Figure 1 illustrates the interaction between user input, application state, and rendering logic during quiz progression.

5.2 Architecture Design

The application follows a fully client-side architecture, as required by the module brief, with all functionality executed in the browser using

JavaScript and no backend services. A centralised state object was implemented to manage application data, including current language, selected category and difficulty, questions index, user answers, and timer state, ensuring all components remain synchronised and simplifying state updates across views.

5.3 Data Design (JSON Structure)

Quiz data is structured using JSON, a lightweight data-interchange format widely used in web applications (MDN Web Docs, 2024a).

Each quiz consists of:

- categories
- questions
- answer options
- correct answer index
- difficulty level

This structure was chosen because JSON is lightweight, human-readable, and aligns with the module requirement for JSON input/output. It also enables the custom quiz upload feature, allowing users to create and load their own quizzes without modifying the application code.

Validation rules were implemented to enforce structural integrity, ensuring every question contains exactly four options, answer indices are valid, and difficulty values are consistent. This prevents malformed data from compromising system stability.

5.4 User Interface Design

The user interface was designed to be clear, structured, and predictable. Key decisions included maintaining a consistent layout across all views so users can anticipate control placement, providing visual feedback to communicate outcomes immediately, and ensuring timer visibility to support user awareness. Readable typography and spacing improve clarity across screen sizes, while a light/dark theme toggle and sound feedback were included to improve comfort and interactivity.

5.5 Interaction and Navigation Design

The application follows a linear navigation model, guiding the user through the following stages:

1) Language selection

2) Category selection
3) Difficulty selection
4) Quiz questions
5) Results and leaderboard

This structured flow ensures that users cannot skip essential steps, reducing the likelihood of errors.

A back navigation feature allows users to revisit previous questions, with safeguards implemented to preserve previously selected answers and prevent unintended state resets.

5.6 Design Decisions and Justification

Two key design decisions were made to balance usability and challenge. First, question quantity varies by difficulty with easier quizzes containing more questions while harder quizzes fewer. This is intended to manage cognitive load and prevent user fatigue without reducing the sense of challenge. Second, a fixed countdown timer was introduced per question to encourage consistent pacing and ensure fairness, as all users are given identical conditions. The visual timer display further supports user awareness and interaction.

6. Implementation

6.1 Overview

Trivia Fiesta was implemented using HTML, CSS and JavaScript in accordance with module requirements, following a Single Page Application model where views are dynamically rendered without page reloads. The system was developed in a modular manner, separating state management, UI rendering and data handling into distinct components to improve maintainability and scalability. Key code snippets are included here, and the full source code is available in the GitHub repository: https://github.com/atheodorou13/COMP1004-Aspa-Theodorou

Detailed data structures are provided in Appendix B.

6.2 Application State Management

A centralised state object was implemented to manage the application's data and control flow. As shown in Figure 2, this ensures

consistency across different views and simplifies updates during user interaction.

```
const state = {
    language: "en",
    data: null,
    categoryId: null,
    difficulty: null,
    qIndex: 0,
    answers: [],
    answered: false,
    questions: [],
    timer: null,
    timeLeft: 15,
    timeout: false,
    soundOn: true
};
```

Figure 2: Centralised state object used for managing quiz state

This structure tracks quiz progress, user choices and UI state in a single location, reducing complexity and preventing inconsistencies.

6.3 DOM Helper Utility

A reusable helper function was implemented to dynamically create DOM elements, avoiding the use of innerHTML. Unlike innerHTML, which inserts raw HTML and risks Cross-Site Scripting (XSS) vulnerabilities, the helper function shown in Figure 3 constructs elements using the DOM API. This is particularly important when handling user-generated content, as it safely attaches event listeners, ensures consistent element creation without unintended side effects.

```javascript
function el(tag, attrs = {}, children = []) {
    const node = document.createElement(tag);

    for (const [k, v] of Object.entries(attrs)) {
        if (k === "className") {
            node.className = v;
            continue;
        }

        // Support both onClick and onclick, etc.
        if ((k.startsWith("on") || k.startsWith("on")) && typeof v === "function") {
            const eventName = k.slice(2).toLowerCase();
            node.addEventListener(eventName, v);
            continue;
        }

        // Handle common DOM properties safely
        if (k in node && typeof v !== "object") {
            node[k] = v;
            continue;
        }

        node.setAttribute(k, String(v));
    }

    for (const child of children) {

        if (child === null || child === undefined) continue;

        if (typeof child === "string") {
            node.appendChild(document.createTextNode(child));
        } else if (child instanceof Node) {
            node.appendChild(child);
        }

    }

    return node;
}

function setView(container, viewNode) {
    container.replaceChildren(viewNode);
}
```

Figure 3: DOM helper function used to safely create and manage dynamic UI element

The setView() function complements the helper by enabling SPA navigation, replacing the current view using replaceChildren() without triggering a page reload.

6.4 Question Handling and Logic

The renderQuestionView() function forms the core of the application, dynamically rendering each question and its answer options based on the current state. As shown in Figure 4, labels are generated bilingually, switching between Latin characters (A-D) and Greek characters (A-Δ) depending on the selected language.

```javascript
function renderQuestionView() {

    const questions = state.questions;
    const q = questions[state.qIndex];

    const labels = state.language === "gr"
        ? ["Α", "Β", "Γ", "Δ"]
        : ["A", "B", "C", "D"];

    const optionsList = el("div", { className: "answers" },
        q.options.map((opt, idx) => {
            const isSelected = state.answers[state.qIndex] === idx;
            const isCorrect = idx === q.answerIndex;

            let cls = "answer";
            const alreadyAnswered = state.answers[state.qIndex] !== undefined;

            if (alreadyAnswered) {

                if (isCorrect) cls += " correct";
                else if (isSelected) cls += " wrong";
                else cls += " disabled";
            }

            return el("button", {
                className: cls,
                type: "button",
                onClick: () => {
                    if (state.answers[state.qIndex] !== undefined) return;

                    state.answers[state.qIndex] = idx;
                    state.answered = true;

                    if (state.soundOn) {
                        if (idx === q.answerIndex) {
                            correctSound.currentTime = 0;
                            correctSound.play();
                        } else {
                            wrongSound.currentTime = 0;
                            wrongSound.play();
                        }
                    }

                    renderQuestionView();
                }
            }, [
                el("span", { className: "answer-label" }, [labels[idx]]),
                el("span", { className: "answer-text" }, [opt])
            ]);
        })
```

Figure 4: Question rendering logic showing dynamic UI updates, answer handling, and feedback mechanisms

Answer buttons are styled conditionally upon selection. Correct answers are highlighted green, incorrect selections red, and remaining options disabled, providing immediate visual feedback. A guard condition prevents multiple selections per question, ensuring fair scoring. User selections are stored in the state.answers array, allowing previously answered questions to be revisited without data loss.

6.5 Custom Quiz Upload and Validation

A custom quiz upload feature was implemented to allow users to load their own quiz data in JSON format. As shown in Figure 5, the upload process reads and validates external data before integrating it into the application state.

```
/* ---------- Load custom quiz JSON ---------- */
function loadCustomQuiz(file) {
    const reader = new FileReader();

    reader.onload = function (event) {
        try {
            const parsed = JSON.parse(event.target.result);

            validateQuizStructure(parsed);

            // Detect quiz language from category name
            const firstCategory = parsed.categories[0]?.name || "";

            if (/[Α-Ωα-ω]/.test(firstCategory)) {
                state.language = "gr";
            } else {
                state.language = "en";
            }

            state.data = parsed;
            state.categoryId = null;
            state.qIndex = 0;
            state.answers = [];
            state.answered = false;

            alert(t("quizLoaded"));
            renderCategoryView();

        } catch (err) {
            console.error(err);
            alert(t("invalidQuiz") + "\n\n" + err.message);
        }
    };

    reader.readAsText(file);
}
```

Figure 5: Custom quiz upload and validation process showing JSON parsing, structure validation, and automatic language detection

The FileReader API reads the uploaded file asynchronously, ensuring responsiveness. Parsed data is passed to validateQuizStructure(), which checks for valid categories, correct question formatting, exactly four answer options, valid answer indices, and recognised difficulty values. Error handling via try-catch ensures invalid files do not cause application failure.

Automatic language detection is implemented using a Greek character regex (/[A-Ωα-ω]/), allowing uploaded quizzes to switch the interface language automatically.

Interface and validation screenshots of the custom quiz upload feature are provided in Appendix E.

6.6 Local Storage Leaderboard

The browser's Web Storage API is used to persist leaderboard data locally across sessions, without requiring a backend (MDN Web Docs, 2024b). As shown in Figure 6, scores are stored in localStorage to maintain user progress.

```
function saveScore(entry) {
    const existing = JSON.parse(localStorage.getItem("triviaScores") || "[]");

    existing.push(entry);

    // Sort by highest score first
    existing.sort((a, b) => b.score - a.score);

    // Keep only top 5
    const topFive = existing.slice(0, 5);

    localStorage.setItem("triviaScores", JSON.stringify(topFive, null, 2));
}
```

Figure 6: Local storage leaderboard implementation showing score persistence, sorting, and top-five filtering

Each entry stores the user's name, score, category, and timestamp. Scores are sorted in descending order and limited ti five entries to maintain a concise leaderboard.

6.7 JSON Score Export

The export feature allows users to download leaderboard data as a JSON file for external use. As shown in Figure 7, data is converted into a downloadable file using client-side processing.

```
function exportScores() {

    const scores = JSON.parse(localStorage.getItem("triviaScores") || "[]");

    if (scores.length === 0) {
        alert("No scores available to export.");
        return;
    }

    const blob = new Blob([JSON.stringify(scores, null, 2)], {
        type: "application/json"
    });

    const url = URL.createObjectURL(blob);

    const a = document.createElement("a");
    a.href = url;
    a.download = "trivia-scores.json";
    a.click();

    URL.revokeObjectURL(url);
}
```

Figure 7: JSON score export process showing conversion of leaderboard data into a downloadable file using the Blob API

The Blob API generates a downloadable file via a temporary object URL enabling export without server-side processing. This complements the upload feature, demonstrating support for structured JSON data for both input and output.

7. Testing

Testing was conducted throughout development using functional, validation, and usability approaches. Functional testing was carried out manually against the requirements defined In Section 4.1, covering navigation, timer behaviour, score calculation, and leaderboard storage, with all high-priority requirements confirmed as passing.

Validation testing targeted the custom JSON upload feature, verifying that validateQuizStructure() rejected malformed files, including missing categories, invalid answer indices, and incorrect option counts, producing appropriate error messages without causing application failure.

Usability testing conducted during a university-organised session identified an edge case where navigating backwards multiple times allows previously answered questions to be reattempted. This was resolved by refining state persistence logic, demonstrating the value of real-user testing in surfacing issues that developer testing had not identified. A full test plan and results are provided in Appendix C.

8. Evaluation and Conclusion

Trivia Fiesta successfully meets the objectives defined at the outset, delivering a fully functional browser-based quiz system within a client-side architecture. The SPA approach enables seamless navigation without page reloads, while the centralised state object ensures consistent data management across all views.

The custom quiz upload feature allows users to load their own content without modifying the application code. Structured validation ensures stability when handling user-generated data, and automatic language detection extends bilingual support beyond the default quiz content. JSON score export satisfies the module requirement for data output, demonstrating consistent handling of structured data.

Usability testing proved valuable in identifying a state management edge case that developer testing had not revealed, reinforcing the importance of real-user feedback. The bilingual implementation also introduced complexity in maintaining consistent translations across interface elements and validation messages.

Several challenges were encountered, particularly in relation to navigation logic and edge-case handling. Ensuring that previously answered questions could not be modified required refinement of the state management approach, while robust validation was needed to handle malformed JSON inputs. These issues were resolved through iterative testing and debugging, improving system reliability.

Key limitations include localStorage restricting leaderboard data to a single browser and device, and the absence of a backend preventing features such as shared leaderboards or user accounts. Future development could address these through backend integration, alongside enhanced accessibility features and expanded question formats.

Overall, the project demonstrates a strong application of front-end development principles, structured data handling, and Agile methodology.

A Project Reflective Review is also provided on the Pebble pad in the designated form.

9. References

Kahoot! (n.d.) Available at: https://kahoot.com (Accessed: 03 January 2026).

Wayground (2025) Available at: https://wayground.com (Accessed: 03 January 2026).

Sporcle (n.d.) Available at: https://www.sporcle.com (Accessed: 03 January 2026).

Pressman, R. (2014) *Software Engineering: A Practitioner's Approach*. 8th edn. New York: McGraw-Hill.

Schwaber, K. and Sutherland, J. (2020) *The Scrum Guide*. Available at: https://scrumguides.org (Accessed: 23 March 2026).

Flanagan, D. (2020) *JavaScript: The Definitive Guide*. 7th edn. Sebastopol: O'Reilly Media.

MDN Web Docs (2024a) *JSON*. Available at: https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON (Accessed: 23 March 2026).

MDN Web Docs (2024b) *Web Storage API*. Available at: https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API (Accessed: 23 March 2026).

APPENDIX A – Sprint Plan and Task Tracker

This appendix presents the sprint plan used during development. The project followed an iterative approach, with tasks organised into nine sprints covering planning, implementation, testing, and finalisation.

Task	Description	Status	Start Date	End Date
COMP1004 - Trivia Fiesta	Trivia Fiesta is a browser-based Single Page Application (SPA) quiz system developed using vanilla HTML, CSS and JavaScript.	Complete	07/10/2025	30/04/2026
SPRINT 1 - Initiation	Project idea definition, project scope planning, and creation of the project initiation document outlining objectives, risks, and development approach.	Complete	07/10/2025	21/10/2025
Define project idea and scope		Complete	07/10/2025	21/10/2025
Identify functional and non-functional requirements		Complete	07/10/2025	21/10/2025
Plan development approach using Scrum and Waterfall		Complete	07/10/2025	21/10/2025
Write project initiation document		Complete	07/10/2025	21/10/2025
SPRINT 2 - Requirements & Planning	Definition of quiz features, creation of the JSON structure for quiz data, collection of bilingual question examples, and initial planning of the user interface.	Complete	22/10/2025	04/11/2025
Define quiz features (categories, scoring, bilingual support)		Complete	22/10/2025	04/11/2025
Design JSON structure for quiz data		Complete	22/10/2025	04/11/2025
Gather example questions in English and Greek		Complete	22/10/2025	04/11/2025
Plan UI layout		Complete	22/10/2025	04/11/2025
SPRINT 3 - Initial Implementation	Creation of the project repository and initial application structure, including HTML layout, SPA container setup, helper functions, and project documentation.	Complete	05/11/2025	19/11/2025
Set up project repository		Complete	05/11/2025	19/11/2025
Build HTML layout		Complete	05/11/2025	19/11/2025

Implement SPA containers		Complete	05/11/2025	19/11/2025
Add helper function		Complete	05/11/2025	19/11/2025
Write READ.ME		Complete	05/11/2025	19/11/2025
SPRINT 4 - Core Quiz Functionality	Implementation of the main quiz logic including category selection, question rendering, answer validation, and score tracking.	Complete	20/11/2025	11/12/2025
Category selection system		Complete	20/11/2025	11/12/2025
Question rendering		Complete	20/11/2025	11/12/2025
Answer selection logic		Complete	20/11/2025	11/12/2025
Score tracking		Complete	20/11/2025	11/12/2025
SPRINT 5 - Mid Project Submission	Preparation of mid-project deliverables including the personal development paragraph, presentation, and submission of the mid-project video demonstration.	Complete	12/12/2025	09/01/2026
Personal development paragraph		Complete	12/12/2025	19/12/2025
Powerpoint		Complete	12/12/2025	19/12/2025
Christmas Break		Complete	20/12/2025	03/01/2026
Mid project video submission		Complete	04/01/2026	09/01/2026
SPRINT 6 - Advanced Features	Development of additional interactive features including the countdown timer, sound effects, theme toggle (light/dark mode), and difficulty filtering.	Complete	10/01/2026	31/01/2026
Timer system		Complete	10/01/2026	31/01/2026
Sound effects		Complete	10/01/2026	31/01/2026
Light / Dark mode		Complete	10/01/2026	31/01/2026
Difficulty filtering		Complete	10/01/2026	31/01/2026
SPRINT 7 - Custom Quiz Feature	Implementation of custom quiz functionality allowing users to download a JSON template and upload their own quizzes with validation checks.	Complete	01/02/2026	15/02/2026
JSON template download		Complete	01/02/2026	15/02/2026
Custom quiz upload		Complete	01/02/2026	15/02/2026

Error messages		Complete	01/02/2026	15/02/2026
JSON structure validation		Complete	01/02/2026	15/02/2026
SPRINT 8 - Testing & Refinement	System testing and interface improvements including bug fixes, UI adjustments, validation testing, leaderboard functionality, and navigation corrections.	Complete	16/02/2026	02/03/2026
Bug fixes		Complete	16/02/2026	02/03/2026
UI improvements		Complete	16/02/2026	02/03/2026
Validation testing		Complete	16/02/2026	02/03/2026
Leaderboard functionality		Complete	16/02/2026	02/03/2026
Navigation fixes (back button / timer reset)		Complete	16/02/2026	02/03/2026
SPRINT 9 - Finalisation	Final project refinement including JSON score export, code commenting, poster creation, report writing, final testing, and submission preparation.	Complete	03/03/2026	30/04/2026
JSON score export		Complete	03/03/2026	17/03/2026
Final code comments		Complete	03/03/2026	17/03/2026
Final UI polishing		Complete	03/03/2026	17/03/2026
Create poster		Complete	18/03/2026	29/03/2026
Write report		Complete	18/03/2026	29/03/2026
Final application test		Complete	18/03/2026	29/03/2026
Spring/Easter break		Complete	30/03/2026	17/04/2026
Submission checklist		Complete	18/04/2026	30/04/2026

APPENDIX B – Data Design

This appendix presents the data structures used within the Trivia Fiesta application. The system relies on structured JSON data and a centralised state object to manage quiz content, user interaction, and application state.

JSON STRUCTURE

```
{
  "categories": [
    {
      "id": "general",
      "name": "General Knowledge",
      "questions": [
        {
          "id": 1,
          "text": "What is the capital city of Canada?",
          "options": [
            "Ottawa",
            "Toronto",
            "Vancouver",
            "Montreal"
          ],
          "answerIndex": 0,
          "difficulty": "easy"
        }
      ]
    }
  ]
}
```

This structure represents quiz data stored in JSON format. Each category contains multiple questions, with each question including a text prompt, four answer options, the correct answer index, and a difficulty level. The application supports both English and Greek quiz datasets, which follow the same JSON structure.

APPLICATION STATE OBJECT

```
const state = {
language: "en",
data: null,
categoryId: null,
difficulty: null,
qIndex: 0,
answers: [],
answered: false,
questions: [],
timer: null,
timeLeft: 15,
timeout: false,
soundOn: true
};
```

The state object stores key application data, including user selections, quiz progress and timer state, ensuring consistency across all views of the application.

LOCAL STORAGE STRUCTURE

```
[
 {
"name": "User",
"score": 5,
"category": "General Knowledge",
"date": "2026-03-20"
 }
]
```

Leaderboard data is stored in localStorage as an array of objects, each containing the user's name, score, category and timestamp. This enables data persistence across sessions without requiring a backend.

All data processing is performed client-side using JavaScript. Quiz data is loaded dynamically from JSON, user interactions are managed through the state object, and leaderboard data is stored using localStorage.

APPENDIX C – Test Plan

Testing Overview

A comprehensive testing strategy was implemented to ensure the reliability, functionality, and usability of the Trivia Fiesta application. Testing was conducted iteratively throughout development, allowing issues to be identified and resolved early. The approach combined verification and validation techniques, ensuring that the system was developed according to its specifications (verification) and met the needs of its intended users (validation). Emphasis was placed on testing core interactive features such as quiz navigation, answer selection, timer behaviour, and score calculation, as these are central to the user experience. In addition, validation testing ensured that invalid inputs, particularly within the custom quiz upload feature, were handled safely without causing system failures.

Test Plan

The test plan was designed to cover multiple levels of testing, including functional, usability, validation, and cross-browser testing. Functional testing verified that all major features operated as expected under normal conditions, while validation testing ensured that incorrect or incomplete inputs were handled appropriately. Testing also ensured that key components, including the application state, user interface, and localStorage, interacted correctly and maintained consistency across the application. Usability testing was conducted informally to assess the clarity and responsiveness of the interface, particularly in relation to bilingual support and user feedback. Cross-browser and device testing confirmed consistent performance across modern browsers and mobile devices. The detailed test cases presented in this appendix demonstrate a structured approach, with clearly defined scenarios, expected outcomes, and results, ensuring coverage of both typical and edge-case behaviours.

Testing Reflection

Several test cases initially resulted in failures, particularly in relation to navigation logic and edge-case conditions. These issues were identified during testing and subsequently resolved, improving overall system reliability.

Quiz Flow and Setup						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC1	SPA	Functional	Select English Language	User clicks English button	UI updates to English (default language applied)	Pass
TC2	SPA	Functional	Select Greek Language	User clicks Greek button	UI updates to Greek	Pass
TC3	SPA	Functional	Select Category	User selects a category	Category stored in state	Pass
TC4	SPA	Functional	Select Difficulty	User selects difficulty	Difficulty stored in state	Pass
TC5	SPA	Functional	Start Quiz	User completes setup	First question displayed	Pass

Question Handling and Logic						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC6	SPA	Functional	Display Question	Question loads from dataset	Question text and options show	Pass
TC7	SPA	Functional	Select answer	User selects an option	Answer stored in state.answers	Pass
TC8	SPA	Functional	Prevent multiple answers	User selects an answer and attempts to select another	Second selection is ignored and original answer remains	Pass
TC9	SPA	Functional	Correct answer feedback	Select correct answer	Option highlighted green	Pass
TC10	SPA	Functional	Incorrect answer feedback	Select incorrect answer	Option highlighted red	Pass
TC11	SPA	Functional	Disable other options	Answer selected	Other options disabled	Pass
TC12	SPA	Functional	Next question	Click “Next” button	Next question loads	Pass
TC13	SPA	Functional	Last question	Reach final question	Results screen displayed	Pass
TC14	SPA	Functional	Back navigation issue	User navigates back	Answer can be changed	Fail
TC15	SPA	Functional	Back navigation fixed	User navigates back	Answer cannot be changed	Pass

Timer Functionality						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC16	SPA	Functional	Timer starts	Question loads	Timer begins countdown	Pass
TC17	SPA	Functional	Timer countdown	Timer running	Timer decreases correctly	Pass
TC18	SPA	Functional	Timer expires	Timer reaches 0 before user answers	Question is marked incorrect	Pass
TC19	SPA	Functional	Timer stops after answer	User selects an answer before timer ends	Timer stops immediately after selection	Pass
TC20	SPA	Functional	Timer edge condition issue	Answer selected at exact timeout	Inconsistent behaviour occurs	Fail
TC21	SPA	Functional	Timer edge condition fixed	Answer selected at exact timeout	Correct behaviour enforced consistently	Pass

Score and Results						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC22	SPA	Functional	Calculate Score	Quiz completed	Score calculated correctly	Pass
TC23	SPA	Functional	Display results	Quiz ends	Score shown to user	Pass
TC24	SPA	Functional	Score accuracy	Multiple correct answers	Score matches correct answers	Pass

Custom Quiz Upload						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC25	SPA	Functional	Upload valid JSON	Upload correctly structured file	Quiz loads successfully	Pass
TC26	SPA	Validation	Invalid JSON format	Upload malformed JSON file	Error message displayed	Pass
TC27	SPA	Validation	Missing options	Upload JSON with <4 options	Error shown	Pass
TC28	SPA	Validation	Invalid answerIndex	answerIndex outside 0-3	Error shown	Pass
TC29	SPA	Validation	Missing category	JSON missing category field	Error shown	Pass
TC30	SPA	Validation	Partial JSON issue	Missing questions array	Application fails to load	Fail
TC31	SPA	Validation	Partial JSON fixed	Missing questions handled	Error handled gracefully with message	Pass
TC32	SPA	Functional	Greek detection	Upload Greek quiz	Language switches to Greek	Pass
TC33	SPA	Functional	English detection	Upload English quiz	Language remains English	Pass

Export Functionality						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC34	SPA	Functional	Export scores JSON	Click export button	JSON file downloaded	Pass
TC35	SPA	Functional	File content check	Open exported file	Data correctly formatted	Pass

UI and Usability						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC36	SPA	Usability	Button feedback	Hover/click buttons	Visual feedback displayed	Pass
TC37	SPA	Usability	Navigation clarity	Move through quiz	UI is user-friendly and easy to navigate	Pass
TC38	SPA	Usability	Language switching UI	Switch languages	UI updates consistently	Pass

Local Storage and Leaderboard						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC39	SPA	Functional	Save score	User enters name and saves	Score stored in localStorage	Pass
TC40	SPA	Functional	Retrieve leaderboard	Reload app	Stored scores displayed	Pass
TC41	SPA	Functional	Score sorting	Multiple scores saved	Sorted in descending order	Pass
TC42	SPA	Functional	Limit entries	More than 5 scores saved	Only top 5 retained	Pass

Theme and Sound Controls						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC43	SPA	Functional	Toggle theme	User clicks theme toggle button	UI switches between light/dark mode	Pass
TC44	SPA	Functional	Theme persistence	User toggles theme and reloads page	Selected theme remains applied	Pass
TC45	SPA	Functional	Toggle sound on	User enables sound	Audio feedback plays on answer selection	Pass
TC46	SPA	Functional	Toggle sound off	User disables sound	No audio plays on answer selection	Pass
TC47	SPA	Functional	Sound toggle during quiz	User toggles sound mid-quiz	Change applies immediately	Pass
TC48	SPA	Validation	Sound disabled state	Sound is off and answer selected	No audio triggered under any condition	Pass

Cross-browser and Device Testing						
Test Case ID	**Page**	**Testing Type**	**Test Scenario**	**Test Description**	**Expected Outcome**	**Result**
TC49	SPA	Cross-browser	Chrome	Run app in Chrome	Works correctly	Pass
TC50	SPA	Cross-browser	Firefox	Run app in Firefox	Works correctly	Pass
TC51	SPA	Cross-browser	Edge	Run app in Edge	Works correctly	Pass
TC52	SPA	Cross-browser	Mobile	Test on smartphone	Layout responsive	Pass

APPENDIX D – UML Diagrams

This appendix presents the UML diagrams used to model the structure and behaviour of the Trivia Fiesta application. These diagrams illustrate user interaction, system flow, data structure, and component interaction within the system.

Figure D1: Use Case Diagram illustrating user interactions with the system

This diagram shows how users interact with the application, including selecting quiz options, answering questions, and viewing results. Additional features such as saving scores, exporting results, and uploading custom quizzes are also represented.

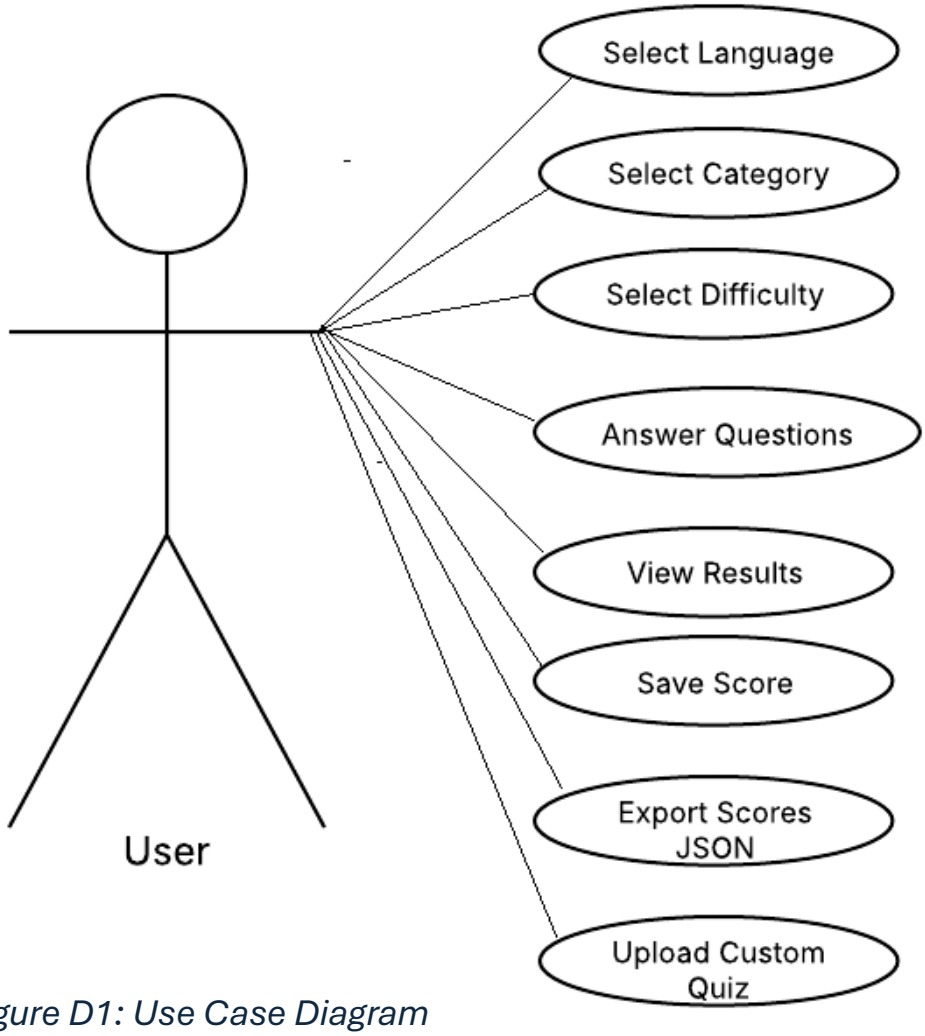

Figure D1: Use Case Diagram

FIGURE D2: Activity Diagram showing the overall quiz flow

This diagram illustrates the sequence of actions within the quiz, from initial setup to question answering and final results. It includes decision points such as timer expiration and question progression.

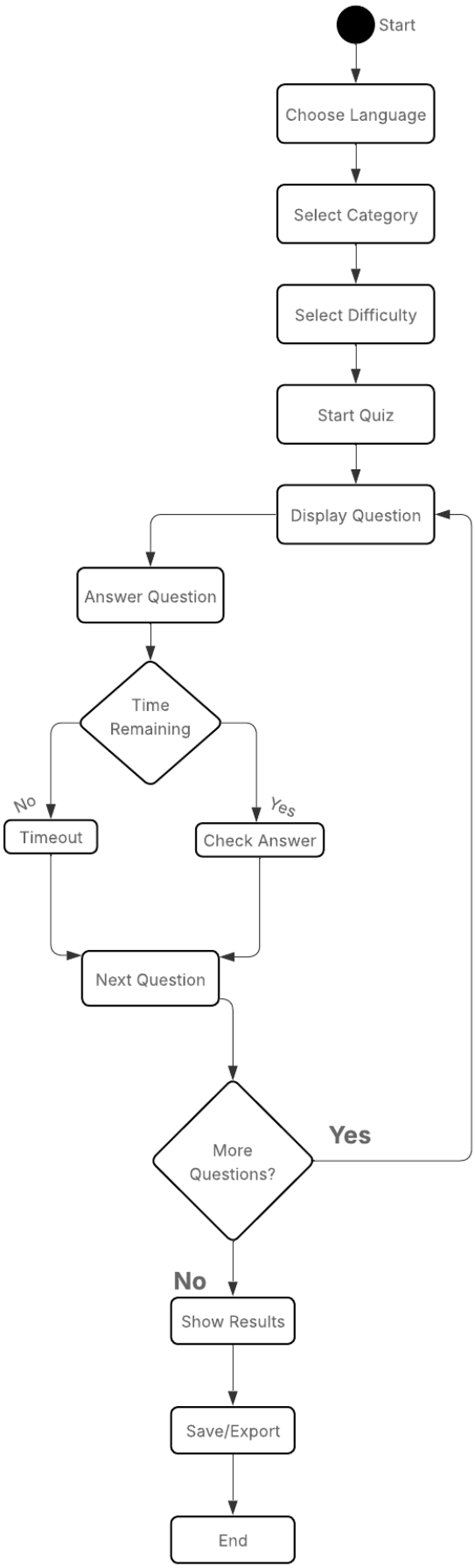

Figure D2: Activity Diagram

FIGURE D3: Sequence Diagram

This diagram demonstrates how user input is processed by the system, showing the interaction between the user interface, application state, and DOM updates during question answering.

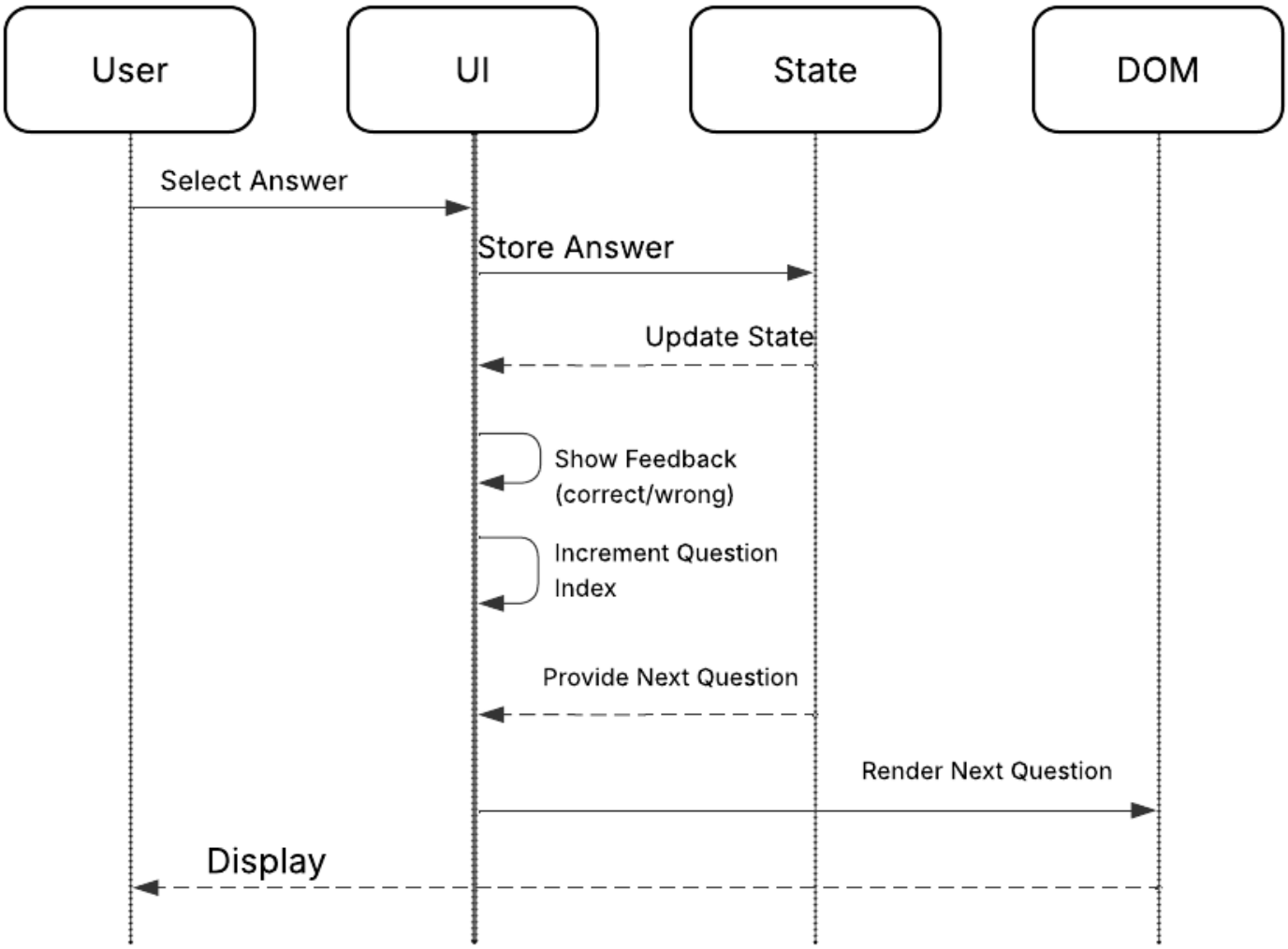

Figure D3: Sequence Diagram

FIGURE D4: Class Diagram

This diagram represents the structure of the application's data model, highlighting the centralised state object and its relationship with question data.

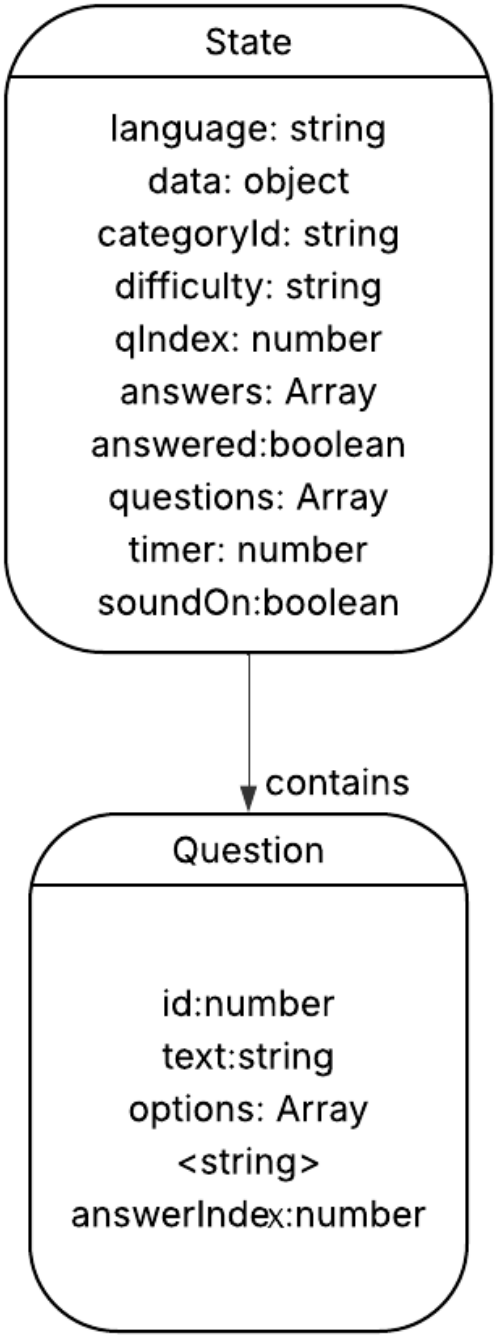

Figure D4: Class Diagram

APPENDIX E - User Interface and Validation Feedback

This appendix presents screenshots demonstrating user interface feedback and validation handling within the Trivia Fiesta application. These examples highlight successful data processing, error handling, and bilingual support across both English and Greek interfaces. These screenshots support the validation testing discussed in Appendix C.

Figure E1: Successful custom quiz upload (English)

Displays confirmation message after a valid JSON quiz file is uploaded successfully.

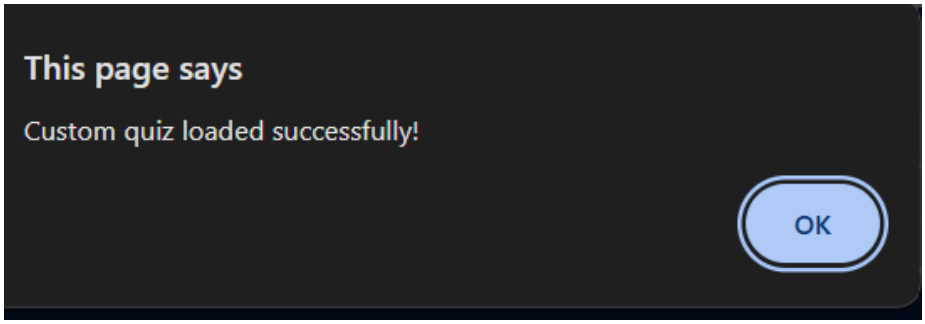

Figure E1 Confirmation message displayed after a valid JSON quiz file is successfully uploaded and processed.

Figure E2: JSON validation error (English)

Shows error handling when a question does not contain exactly four options, preventing invalid data from being loaded.

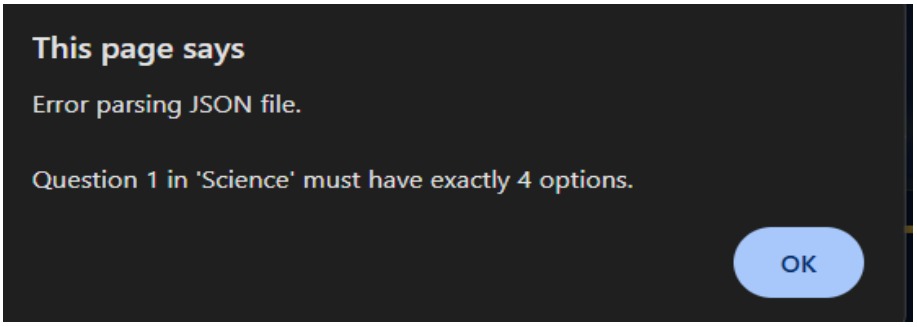

Figure E2: Error message displayed when uploaded quiz data does not meet structural requirements, preventing invalid input from being processed.

Figure E3: Successful custom quiz upload (Greek)

Demonstrates bilingual support, with confirmation messages displayed in Greek when a Greek quiz file is uploaded.

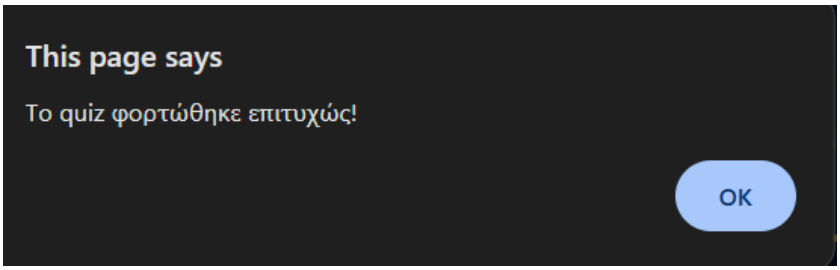

Figure E3: Confirmation message displayed in Greek following successful upload and processing of a valid quiz file.

Figure E4: JSON validation error (Greek)

Shows validation feedback in Greek, ensuring consistent user experience across both supported languages.

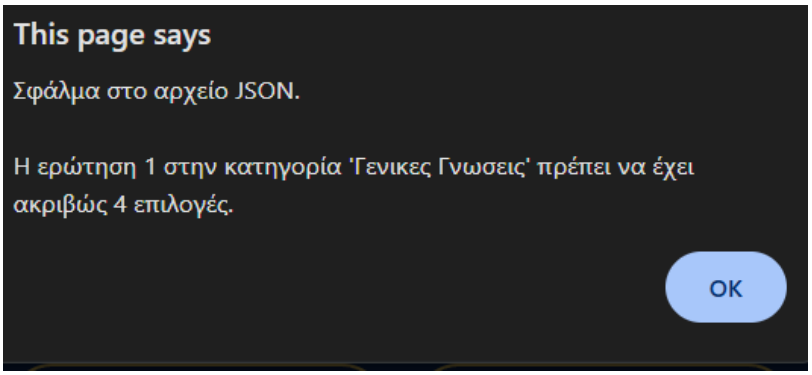

Figure E4: Validation error message displayed in Greek when uploaded quiz data contains structural issues, ensuring consistent bilingual feedback.

APPENDIX F – AI Declaration Form

Student Declaration of AI Tool use in this Assessment

Please indicate your level of usage of generative AI for this assessment - please tick the appropriate category(s).

If the "Assisted Work" or "Partnered Work" category is selected, please expand on the usage and in which elements of the assignment the usage refers to.

Solo Work	**S1 - Generative AI tools have not been used for this assessment.**	☐
Assisted Work	**A1 – Idea Generation and Problem Exploration** Used to generate project ideas, explore different approaches to solving a problem, or suggest features for software or systems. Students must critically assess AI-generated suggestions and ensure their own intellectual contributions are central.	☐
	A2 - Planning & Structuring Projects AI may help outline the structure of reports, documentation and projects. The final structure and implementation must be the student's own work.	☐
	A3 – Code Architecture AI tools maybe used to help outline code architecture (e.g. suggesting class hierarchies or module breakdowns). The final code structure must be the student's own work.	☐
	A4 – Research Assistance Used to locate and summarise relevant articles, academic papers, technical documentation, or online resources (e.g. Stack Overflow, GitHub discussions. The interpretation and integration of research into the assignment remain the student's responsibility.	☒
	A5 - Language Refinement Used to check grammar, refine language, improve sentence structure in documentation not code. AI should be used only to provide suggestions for improvement. Students must ensure that the documentation accurately reflects the code and is technically correct.	☒
	A6 – Code Review AI tools can be used to check comments within the code and to suggest improvements to code readability, structure or syntax. AI should be used only to provide suggestions for improvement. Students must ensure that the code accurately reflects their knowledge and is technically correct.	☐
	A7 - Code Generation for Learning Purposes Used to generate example code snippets to understand syntax, explore alternative implementations, or learn new programming paradigms. Students must not submit AI-generated code as their own and must be able to explain how it works.	☐

	A8 - Technical Guidance & Debugging Support AI tools can be used to explain algorithms, programming concepts, or debugging strategies. Students may also help interpret error messages or suggest possible fixes. However, students must write, test, and debug their own code independently and understand all solutions submitted.	☐
	A9 - Testing and Validation Support AI may assist in generating test cases, validating outputs, or suggesting edge cases for software testing. Students are responsible for designing comprehensive test plans and interpreting test results.	☐
	A10 - Data Analysis and Visualization Guidance AI tools can help suggest ways to analyse datasets or visualize results (e.g. recommending chart types or statistical methods). Students must perform the analysis themselves and understand the implications of the results.	☐
	A11 - Other uses not listed above Please specify:	☐
Partnered Work	**P1 - Generative AI tool usage has been used integrally for this assessment** Students can adopt approaches that are compliant with instructions in the assessment brief. Please Specify:	☐

Please provide details of AI usage and which elements of the coursework this relates to: **Language refinement – used to assist with the word limit and grammar check.** **Research assistant – used to research questions and learn about scrum.**

I understand that the ownership and responsibility for the academic integrity of this submitted assessment falls with me, the student.	☒
I confirm that all details provide above are an accurate description of how AI was used for this assessment.	☒